week 12 V-JEPA

📄 **V-JEPA 2**: Self-Supervised Video Models Enable Understanding, Prediction and Planning
arxiv.org/pdf/2506.09985

1. Abstract

💡 "다음에 무슨 일이 일어날지" 예측하는 능력— **World model**
→ AI도 World model을 비디오만 보고 스스로 학습할 수 있지 않을까? (연구 시작)

▼ wordl model 간략 설명

- 경에 대한 내부 표현을 구축하는 머신러닝 시스템
- 사용자의 행동에 따라 환경이 시간에 따라 어떻게 변화하는지 예측
  - 복잡한 현실 세계의 구조와 동역학을 이해

↔ **Large Language Models (LLMs):** Predict the next word or token based on text patterns (물리적 상태, 움직임 등을 예측하는 부분에서 다름)

2. Introduction

1. 기존 연구 한계

- state-action 시퀀스로 구성된 상호작용 데이터에 의존
- 명시적 보상 피드백까지 필요로 함

  → 실제 상호작용 데이터 부족, 만들기 어려움 — **확장성 제약**
- 최근 연구는 인터넷 규모 비디오 + 상호작용 데이터를 활용해 로봇 제어용 action-conditioned 비디오 생성 모델을 학습

  → 비디오를 화면 그대로 예측 (완전 세밀한 부분까지 맞히려고 함 — 계산 낭비)

  — 계획 능력보다는 **예측의 충실도와 시각적 품질 평가**에 치중하는 경향

2. V-JEPA 2의 해결

> **"쓸데없는 디테일은 무시하고, 중요한 것만 예측하자"**

❌ 배경의 나뭇잎 하나하나 움직임 → 안 중요함, 무시

✅ 공이 어디로 움직이는지 큰 흐름 → 이게 중요, 이것만 예측

- 자기지도학습 가설에 기반해, **관찰만으로 세계에 대한 배경지식 포착**

2단계 학습

단계	내용
1단계 Action-free 사전학습	인터넷 규모 비디오로 마스크-디노이징 특징 예측(mask-denoising feature prediction) 목표 학습. 최대 10억 파라미터, 100만+ 시간 비디오 사용 — 영상 일부를 가려놓고 "이 가려진 부분에 뭐가 있었을지" 맞히는 연습을 시킴 (어떻게 움직이는지, 장면이 어떻게 이어지는지를 자연스럽게 익힘)
2단계 Action-conditioned 사후학습	소량의 상호작용 데이터로 world model(V-JEPA 2-AC) 학습. 3억 파라미터 트랜스포머, block-causal attention, 다음 프레임 표현을 행동+이전 상태 조건으로 자기회귀 예측 — "이런 행동을 하면 다음 장면이 이렇게 될 것이다"를 예측하는 모델 生

3. Related work

World models and planning

- Sutton and Barto (1981), Chatila and Laumond (1985) 등 초창기부터 AI 연구자들은 **세계의 내부 모델**(동역학 모델링 + 정적 환경 매핑)을 이용해 효율적인 계획과 제어를 하는 에이전트를 추구
- 기존 연구는 **시뮬레이션 과제**와 **실세계 이동·조작 과제** 양쪽에서 world model을 탐구
- world model의 예측 공간은 세 갈래로 나뉨:
  1. **픽셀 공간**에서 직접 예측
  2. **학습된 표현 공간**에서 예측
  3. **키포인트(keypoint) 표현** 같은 더 구조화된 표현 공간

기존 접근의 한계

- 실세계 로봇 과제에서 성과를 보인 기존 방법들은 대부분 **과제별(task-specific)** world model을 학습했고, 배치될 환경의 상호작용 데이터에 의존.
  평가도 **탐색한 과제 공간 내 성능**에 초점을 맞췄지, 새로운 환경이나 못 본 물체로의 일반화는 다루지 않음

→ 본 연구는 과제에 무관한(task-agnostic) world model을 학습하고, 새로운 환경·물체로의 일반화를 보인다.

최근 동향과의 비교

- 인터넷 규모 영상 + 상호작용 데이터로 범용 action-conditioned **비디오 생성** 모델을 학습하는 최근 연구들
  : 시각적으로 그럴듯한 계획 영상을 생성하는 능력은 보였지만, **실제로 로봇을 제어하는 데 쓴 적은 D없음**
- 생성 모델을 정책 학습에 통합한 연구들도 있음.
  본 연구는 이와 달리, **전문가 궤적이 필요한 모방학습 단계를 피하고** 모델 예측 제어(MPC)로 세계 모델을 직접 활용하는 것이 목표. (두 접근은 상호 보완적이라 향후 결합 가능)
- **가장 가까운 선행 연구**
  : Zhou et al. (2024), Sobal et al. (2025) — world model을 단계별 또는 end-to-end으로 학습해 계획 과제를 zero-shot으로 푼다는 것을 보임. 다만 이들은 **소규모** 계획 평가에 그쳤음.

→ 본 연구는 같은 원리를 스케일업해서 실세계 로봇 과제에 적용했다는 점에서 차별화

로봇 제어를 위한 Vision-Language-Action(VLA) 모델

- 최근 모방학습 기반 접근들은 인터넷 규모 영상-텍스트로 사전학습된 비디오-언어 모델을 **전문가 시연의 행동 복제(behavior cloning)**로 파인튜닝해 행동을 예측하도록 만들어 상당한 일반화 성과를 보였다.
- **한계**:
  - 세계에 대한 **명시적 예측 모델이 없고**, 추론 시점의 계산을 계획에 활용하지 않아 **학습 데이터에 없던 행동을 예측**할 수 있을지 불분명
  - **고품질 대규모 원격조종 데이터**가 필요하고, **성공한 궤적만** 활용 가능

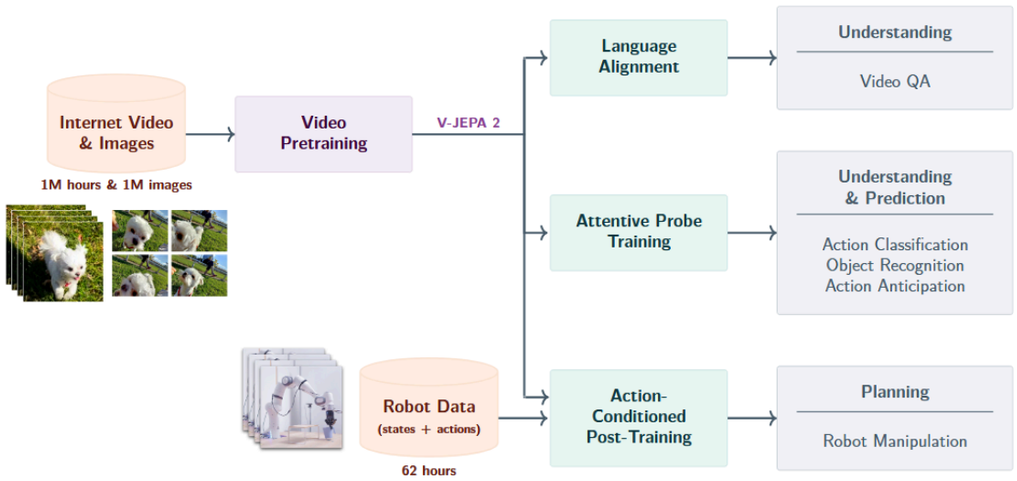
→ **본 연구는 성공/실패 여부와 관계없이 어떤 상호작용 데이터든 활용**하는 것이 목표라는 점에서 대조된다.

시각 파운데이션 모델 (Vision Foundation Models)

- 비디오 파운데이션 모델 연구는 대규모 이미지·영상 관측 데이터셋으로 **범용 시각 인코더**를 학습할 수 있음을 보여왔음 — 이미지 자기지도학습, 비디오 자기지도학습, 약한 언어 지도(weak language supervision), 또는 이들의 조합을 통해.
- **기존 연구의 경향**
  : probe 기반 평가나, LLM과 정렬 후 VQA 과제로 **이해(understanding) 평가**에 주로 초점
- 하지만 시각 시스템의 중요한 목표 중 하나는 에이전트가 **물리 세계와 상호작용**할 수 있게 하는 것 (Gibson, 1979의 생태학적 지각 이론 인용)

→ 본 연구는 시각 이해 과제 결과를 넘어서, 대규모 자기지도 비디오 학습이 새로운 환경에서 zero-shot 계획 과제까지 풀 수 있게 하는지를 탐구

4. Method & Experiments



	설명	결과물
Language Alignment (언어 정렬)	영상 이해 능력에 언어(LLM)를 붙임	영상 보고 질문에 답하기 (Video QA)
Attentive Probe Training (간단한 분류기 얹기)	원래 모델은 그대로 두고, 위에 작은 분류기만 새로 학습	행동 분류, 물체 인식, "다음에 뭘 할지" 예측
Action-Conditioned Post-Training (행동 조건부 추가학습)	로봇 팔이 움직인 영상(62시간)만 추가로 살짝 더 학습	로봇이 물건을 집거나 옮기는 계획 세우기

2. Scaling Self-Supervised Video Pretraining

> **Mask-Denoising in Representation Space** — 가려놓고 맞히기
> + 요약된 특징만 예측!

2.1 방법론

🎯 마스킹된 영상 뷰 x로부터, 원본 영상 y의 학습된 표현 예측

→ 2개 task로 구성

- **인코더 Eθ(·)**: 영상 표현을 추출
- **Predicter Pφ(·)**: 마스킹된 부분의 표현을 예측
  - 표현 붕괴(collapse)를 막기 위해 stop-gradient와 인코더 가중치의 지수이동평균(EMA) 사용

    — 정답이 갑자기 확 바뀌지 않도록 정답 쪽(인코더)은 천천히 업데이트(EMA)

구조

- 인코더, predicter 모두 Vision Transformer(ViT)로 구성
  - RoPE(Rotary Position Embedding) 사용
    — 상대적 위치 정보를 인코딩하기 위해
  - 영상은 "시간(언제) + 세로(어디) + 가로(어디)" 3차원 정보 有. RoPE는 이 위치 정보를 더 안정적으로 알려주는 방법이라, 모델이 커져도 학습이 잘 되게 도와줌

4가지 핵심 확장 요소

1. **데이터 확장**: 200만 → 2,200만 개 비디오
2. **모델 확장**: 3억 → 10억+ 파라미터 (ViT-L → ViT-g)
3. **긴 학습**: warmup-constant-decay 학습률 스케줄 도입, 9만 → 25.2만 이터레이션
4. **고해상도**: warmup·constant 단계에서는 짧고 저해상도로 학습, 마지막 decay 단계에서 해상도·클립 길이 증가

2.2 스케일링 실험 결과

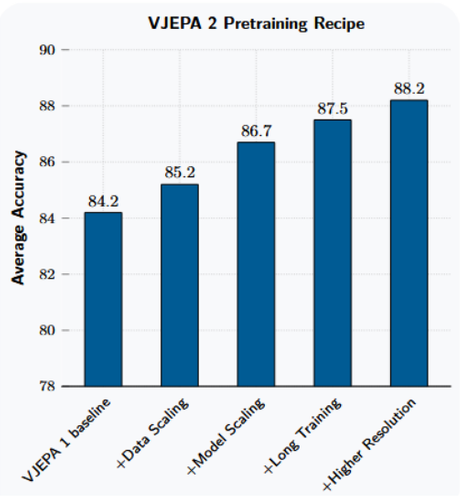


Figure 3 Scaling Ingredients

ViT-L/16, 200만 비디오 기준 모델을 베이스라인으로:

- 데이터 200만→2200만 (VM22M): **+1.0**
- 모델 3억→10억 파라미터 (ViT-g/16): **+1.5**
- 학습 9만→25.2만 이터레이션: **+0.8**
- 해상도(256→384) + 프레임 수(16→64) 동시 증가: 최종 **88.2%**, 베이스라인 대비 누적 **+4.0**

2.3 사전학습 데이터셋

데이터 구성 (VideoMix22M/VM22M, 총 2200만 샘플)

- Something-Something v2 (1인칭 시점)
- Kinetics 400/600/700 (3인칭 행동 영상)
- HowTo100M (유튜브 튜토리얼)
- YT-Temporal-1B (일반 유튜브 영상, YT1B)
- ImageNet (이미지를 16프레임 동일 이미지로 복제해 비디오처럼 취급)

2.4 사전학습 레시피

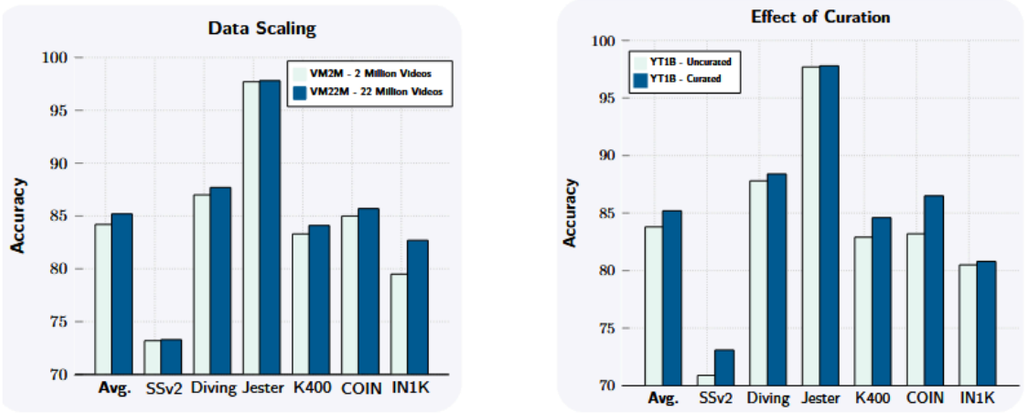


Figure 4 Data Scaling & Curation

- **모델 크기**
  - 3억→10억 파라미터로 **+1.5** (SSv2 +1.6, Kinetics +1.5)
- **학습 스케줄**
  - warmup-constant-cooldown 방식 채택, half-cosine 스케줄과 성능은 비슷하지만 비용 효율적

    ▼ warmup-constant-cooldown

    Warmup-Constant-Cooldown 방식 (이 논문이 쓴 방식)

    이름 그대로 **3단계**로 나뉨.

```
학습률
  ↑
  |    ________________
  |  /                \
  | /                  \
```

```
|/                    \___
+---------------------------→ 시간
 [워밍업]   [일정하게 유지(constant)]  [쿨다운: 서서히 감소]
```

1. **Warmup (워밍업)**: 처음엔 학습률을 0에서 천천히 올림 (급발진 방지)
2. **Constant (일정 유지)**: 학습률을 **한동안 그대로 유지** — 여기가 핵심 차이점!
3. **Cooldown (쿨다운)**: 마지막에만 학습률을 서서히 낮춤

- **점진적 해상도 학습**
  - 64프레임×384×384 직접 학습 시 약 **60 GPU-년** 소요
    → 점진적 해상도 전략으로 **8.4배 GPU 시간 절감**
- **해상도/프레임 확장 효과**
  - 16→64프레임 확장으로 **+0.7**
  - 128·256프레임까지 늘려도 추가 향상 없음 (64프레임이 한계점)

3. V-JEPA 2-AC: Learning an Action-Conditioned World Model

> **행동 → 다음 장면 맞히기 (Planning)**
> V-JEPA 2(masking 부분) — 에이전트가 취할 행동이 미치는 인과적 효과 고려 X

```
[현재 장면] + [로봇이 한 행동(팔을 얼마나 움직였나)] → [다음 장면은?]
```

3.1 방법론 — 행동 조건부 world model 학습

- **목표**
  - 사전학습된 V-JEPA 2로부터,
    폐루프(closed-loop) 모델 예측 제어로 로봇을 제어할 수 있는 **잠재(latent) world model**을 얻는 것
  - 제어 행동과 고유수용성(proprioceptive) 관측을 조건으로 미래 영상 관측의 표현을 자기회귀적으로 예측
- **데이터**
  - Droid 원본 영상 약 **62시간**
    (보통 3~4초짜리 짧은 영상, 7자유도 Franka 팔 + 2지 그리퍼)
  - **"라벨 없는(unlabeled)"** 의 의미
    : 보상, 작업 종류, 성공 여부 메타데이터를 쓰지 않음.
    원본 영상 + 엔드이펙터 상태 신호만 사용
- **모델 입력**

항목	내용
클립	4초 길이, 4초보다 짧은 영상은 버림 (그래서 62시간 미만)
해상도/프레임률	256×256, 4fps → 클립당 **16프레임** (x_k)
엔드이펙터 상태 s_k	7차원: 위치 3 + 방향(오일러 각) 3 + 그리퍼 상태 1
행동 a_k	인접 프레임 간 엔드이펙터 상태의 **변화량** (7차원), k=1~15
증강	random-resize-crop (종횡비 0.75~1.35)

- **손실 함수**
  - V-JEPA 2 인코더 E(·)를 이미지 인코더로 사용
    - 각 프레임을 독립적으로 인코딩
      → $z_k = E(x_k)$ 얻음
    - 인코더는 **고정** — 인코더(눈)는 그대로 고정해두고, 작은 예측기(뇌의 "행동 결과 예측" 부분)만 새로 학습
    - (a_k, s_k, z_k)를 시간 순서대로 교차 배열해 예측기 P_φ에 넣고,
      다음 상태 표현 $\hat{z}_{k+1}$을 예측

      — 시간 순서대로 "행동 → 상태 → 프레임 특징" 묶음을 이어 붙여서, 하나의 긴 토큰 시퀀스로 트랜스포머에 넣는 것

```
[행동 1개 + 상태 1개 + 패치 256개]

[a₁][s₁][z₁의 256개 패치] | [a₂][s₂][z₂의 256개 패치] | [a₃][s₃][z₃의 256개 패치] | ...
└──────── 시점 1 ────────┘ └──────── 시점 2 ────────┘ └──────── 시점 3 ────────┘
```

1. **Teacher-forcing 손실**
   : 정답 이전 프레임을 매번 입력으로 주고 다음 프레임 표현을 예측, L1 거리 평균 (T=15)
2. **Rollout 손실**
   : 추론 시 자기회귀 방식으로 이어서 예측하는 능력을 높이기 위한 손실. 실제로는 **T=2**로, 예측기를 재귀 한 단계만 미분
3. **최종 목적함수**
   : L(φ) = L_teacher-forcing + L_rollout (예측기 가중치 φ에 대해서만 최소화)

▼ Teacher-forcing vs Rollout 손실

	Teacher-forcing	Rollout
방식	매 스텝마다 **진짜 정답 장면**을 보여주고 다음 장면 예측	**자기가 예측한 장면**을 다시 입력으로 써서 이어서 예측
비유	선생님이 매번 정답 풀이를 보여주며 다음 문제를 풀게 함	내가 푼 풀이를 바탕으로 이어서 푸는 연습
필요한 이유	기본기 학습	실전에서는 정답이 없으니, 예측이 누적돼도 안 무너지게 훈련

실전(planning)에서는 모델이 자기 예측을 이어서 쓰며 미래를 상상해야 하기 때문에 rollout 손실이 꼭 필요함



Figure 6 V-JEPA 2-AC training.

▼ Figure 6 설명

공통 구조 (아래에서 위로)

1. **맨 아래**: 로봇 팔 영상 프레임 t=0~4
2. **frame encoder (frozen)**: 각 프레임을 인코딩해 특징 맵 z_0~z_4를 만든다. 고정이라 학습되지 않는다.
3. **interleave action and pose tokens**: 특징 맵 앞에 행동·상태 토큰(진한 회색 작은 칸 2개)을 붙여 한 블록으로 만든다. 지난번에 설명한 교차 배열이에요.
4. **predictor (time-causal attention)**: 이 시퀀스를 받아 다음 시점의 특징 $\hat{z}$를 예측한다. 이 부분만 학습된다.
5. **맨 위**: 예측값 $\hat{z}_1$~$\hat{z}_4$를 정답과 **L1 거리**로 비교한다.

(그림은 T=4로 단순화한 예시이고, 실제 학습은 teacher forcing T=15, rollout T=2예요.)

왼쪽: Teacher Forcing Loss

입력 z_0, z_1, z_2, z_3는 모두 인코더가 만든 진짜 특징

```
진짜 z₀ (+행동) → ẑ₁ 예측  ─┐
진짜 z₁ (+행동) → ẑ₂ 예측   │ 각각 진짜 z₁, z₂, z₃, z₄와 L1 비교
진짜 z₂ (+행동) → ẑ₃ 예측   │ (빨간 박스 = 비교 대상)
진짜 z₃ (+행동) → ẑ₄ 예측  ─┘
```

- 매 스텝마다 정답 프레임을 입력으로 주니, 예측이 틀려도 다음 스텝에 영향을 주지 않음
- **네 개의 다음 프레임 예측을 한 번에** 계산하고 손실도 네 개를 모두 쓴다는 점이 특징
- 학습 신호가 풍부하고 안정적이라 "한 스텝 앞을 예측하는 기본기"를 배우기에 좋음

오른쪽: Rollout Loss

진짜 입력은 z_0 하나뿐이고, 나머지는 모델이 자기가 만든 예측을 다시 입력(보라색 점선 화살표).

```
진짜 z₀ (+행동) → ẑ₁ 예측
  ẑ₁ (+행동) → ẑ₂ 예측   ← 자기 예측을 입력으로!
  ẑ₂ (+행동) → ẑ₃ 예측   ← 또 자기 예측을 입력으로!
  ẑ₃ (+행동) → ẑ₄ 예측
                ↓
       마지막 ẑ₄만 진짜 z₄와 L1 비교
```

- 그림에서 보라색으로 칠해진 칸이 **모델이 만든 예측**이고, 회색 칸이 **인코더가 만든 진짜 특징**.
- 손실은 **맨 마지막 $\hat{z}_4$와 z_4** 하나만 비교 (빨간 박스가 오른쪽엔 하나뿐).
- 정답 없이 자기 예측만으로 여러 스텝을 이어가야 해서, 작은 오차가 쌓이는 상황을 그대로 겪으며 학습

- **구조 (Architecture)**
  - 약 **3억 파라미터** 트랜스포머 (24층, 16헤드, 은닉 차원 1024, GELU)
  - 행동·상태·특징 맵은 각각 별도의 학습 가능한 affine 변환으로 은닉 차원에 매핑, 출력은 다시 인코더 임베딩 차원으로 매핑

    — 행렬을 곱하고 벡터를 더하는 단순한 변환(= Linear layer). 이 논문에서는 서로 크기가 다른 행동·상태·특징을 트랜스포머가 처리할 수 있는 같은 크기로 맞추는 어댑터 역할
  - **3D-RoPE**로 패치의 시공간 위치 표현 (행동·상태 토큰에는 시간축 RoPE만 적용)
  - **Block-causal attention**: 각 시점의 패치 특징은 **같은 시점**의 행동·상태·다른 패치 특징, 그리고 **이전 시점** 정보만 볼 수 있음 (미래는 못 봄)

3.2 planning을 위한 행동 추론

- **에너지 최소화**
  - 목표 상태의 이미지 입력
    → V-JEPA 2-AC를 planning에 활용해 다운 스트림 과제 수행

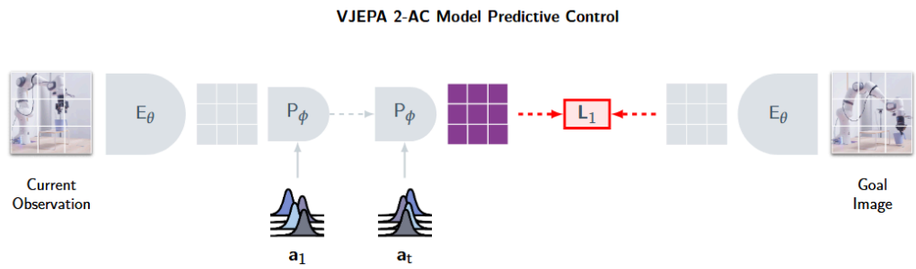
    — 매 시간 스텝마다 목표 조건부 에너지 함수를 최소화하는 고정 길이 구간의 행동 시퀀스 계획
    → 그중 첫 번째 행동 실행, 새로운 상태 관측(이 과정 반복)

$$E(\hat{a}_1 : T; z_k, s_k, z_g) := \| P(\hat{a}_1 : T; s_k, z_k) - z_g \|_1$$

    ▼ 식(5) 설명

    s_k : 현재 엔드이펙터 상태
    x_k : 현재 관측 프레임
    x_g : 목표 이미지

    둘은 같은 비디오 인코더로 따로 인코딩되어 특징 맵 z_k와 z_g가 된다. 계획 구간 T가 주어졌을 때, 다음 목표 조건부 에너지 함수를 최소화해 로봇 행동 시퀀스 (a_i^*)를 구한



▼ figure 7 설명

```
[현재 관측] → Eθ → z_k → Pφ(a₁) → Pφ(aₜ) → 상상한 미래 ẑ ─┐
                                                          L1 (비교)
[목표 이미지] → Eθ → z_g ──────────────────────────────────┘
```

그림 요소	의미
Current Observation → Eθ	현재 카메라 이미지를 인코더로 특징 맵(z_k)으로 변환
Pφ (예측기) 여러 개	같은 예측기를 행동마다 한 번씩 반복 적용 (점선 화살표 = 출력을 다음 입력으로 전달)
a_1, a_t 아래의 종 모양 분포	각 시점의 행동을 **가우시안 분포에서 샘플링**한다는 표시 (여러 겹 = 후보가 여러 개)
보라색 격자	모델이 **상상한** T스텝 뒤 특징 맵 (Figure 6에서도 보라색 = 모델 예측)
Goal Image → Eθ	목표 이미지를 같은 인코더로 특징 맵(z_g)으로 변환
L1 (빨간 박스)	상상한 결과와 목표 특징 사이의 거리 = **에너지**. 이걸 줄이는 게 목표

- 모델은 **모델이 T스텝 뒤로 상상한 상태 표현**과 **목표 표현** 사이의 L1 거리를 최소화하는 궤적을 골라 행동 시퀀스를 추론
  - 실제로는 매 계획 스텝마다 식 (5)를 **Cross-Entropy Method(CEM)**로 최소화하고, 로봇에는 첫 번째 행동만 실행한 뒤 다시 계획한다(receding horizon control).

    → 특징 공간에서 예측하므로 후보마다 영상을 생성하는 것보다 **훨씬 저렴 — 비디오 생성 기반 계획은 비용이 큰 문제를 해결**

    ▼ CEM

    CEM(Cross-Entropy Method)이 어떻게 돌아가나?

    "좋은 행동 시퀀스를 어떻게 찾나?"에 대한 답.
    수학적 최적화 대신 **샘플링** → **선별** → **분포 갱신**을 반복하는 방식

    1. 초기화

    : 각 시점(1~T)의 각 행동 좌표마다 가우시안 분포를 만든다 (평균 0, 분산 1). 그림의 종 모양이 이것이에요.

    2. 샘플링

    : 분포에서 후보 행동 시퀀스를 여러 개 뽑는다.

    3. 상상 & 평가

    : 각 후보를 세계 모델(Pφ)에 넣어 T스텝 뒤 상태를 상상하고, 목표와의 L1 거리(에너지)를 계산한다.

    4. 상위 k개 선별

    : 목표에 가장 가까워진 상위 k개 후보만 남긴다.

    5. 분포 갱신

    : 상위 k개 후보의 평균과 분산으로 가우시안을 다시 세팅한다. 그러면 분포가 "좋았던 행동 근처"로 좁혀진다.

    6. 반복

    : 2~5를 여러 번 반복한다. 분포가 점점 좋은 행동에 수렴한다.

7. 결과 반환

: 마지막에 가우시안의 **평균**을 최종 행동 시퀀스로 선택한다.

8. 실행

: 그중 **첫 번째 행동만** 로봇에 실행한다. 새 관측을 받아 처음부터 다시 계획한다.

CEM를 쓰는 이유

- world model은 신경망이라 **기울기 기반 최적화가 불안정**할 수 있는데, CEM은 예측 결과의 "점수"만 있으면 됨.
- 후보를 병렬로 한꺼번에 평가할 수 있어 GPU에 적합
- 특징 공간에서 예측하므로 후보마다 영상을 생성하는 것보다 **훨씬 저렴**.
  이게 지난번 서론에서 말한 "비디오 생성 기반 계획은 비용이 크다"는 문제를 피하는 지점

4. Zero-shot 로봇 제어

> **"영상 62시간만 보고 학습한 모델이, 한 번도 가본 적 없는 연구실의 로봇 팔을 추가 학습 없이 움직일 수 있는가?"** → 이를 증명하고자 함

4.1 실험 설정

비교 대상(baseline) 2개

	Octo	Cosmos
종류	비전-언어-행동(VLA) 모델	비디오 생성 기반 세계 모델
원래 학습	Open-X Embodiment (100만+ 궤적)	2천만 시간 영상 (행동 정보 없음, latent diffusion 7B)
Droid 적용	전체 Droid 데이터로 **행동 복제(behavior cloning)** 파인튜닝. hindsight relabeling으로 궤적 중 최대 20스텝 앞의 프레임을 목표 이미지로 사용	공식 코드로 **행동 조건부** 파인튜닝. 학습률 하향, 조건 프레임 dropout 제거, 노이즈 수준 e^2배 증가
제어 방식	정책이 행동을 직접 출력	세계 모델 + CEM 계획

4.2 결과

1. **Single-goal Reaching(목표 이미지 한 장으로 팔 끝 옮기기)**
   - 기본적인 행동 이해와 단안 카메라로부터의 **3D 공간 이해** 측정
   - 3가지 도달 과제 모두 목표 위치에서 4cm 이내로 접근,
     오차가 단조롭게 감소 — 시각 서보잉(visual servoing)
   - **라벨 無 실제 영상 데이터로 학습해서 달성**
   - Figure 9
2. **Prehensile Manipulation(물체 조작)**
   - 세부 내용
     - 과제 — Grasp, Reach with object, pick-and-place
     - 성공률은 과제 변형을 석은 10회 시도 평균
     - Grasp, Reach with object : 목표 이미지 1장
     - Pick-and-place : 서브골 이미지 2장 + 최종 목표
       1. 물체를 잡은 장면 → 4스텝 동안 최적화
       2. 목표 위치 근처에 물체가 있는 장면 → 10스텝
       3. 최종 목표 → 마지막 4스텝

     ▼ WHY

     목표 이미지 하나(물체가 도착지에 있는 장면)만 주면 모델은 **당장 목표와 가장 비슷해지는 쪽으로만** 움직이려 함(greedy). 하지만 물건을 옮기려면 일단 **집는 행동**이 먼저 필요해서, 목표에서 멀어지는

듯한 중간 단계를 거쳐야 함.

과제	무엇을 시험하나
Grasp	시각 피드백으로 정밀하게 잡는 제어
Reach with object	물체를 든 채 이동 → 떨어뜨리지 않는 직관적 물리 이해
Pick-and-place	위 기본 기술들의 **조합**

- 결과
  - 모든 모델이 reach는 잘하지만, **물체와 상호작용하는 과제에서 차이**가 뚜렷
  - 성공률은 물체 종류에 따라 달라짐
    (컵: 테두리를 잡아야 해서 정밀도 필요 / 박스: 잡는 방법은 많지만 그리퍼를 충분히 벌려야 함)
  - **V-JEPA 2-AC가 모든 과제에서 최고 성공률**

3. **V-JEPA 2 -AC vs Cosmos (계획 시간 비교)**
   - 둘 다 CEM 사용, GPU는 RTX 4090 한 장
   - **Cosmos**
     - 샘플 80개, 정제 10회, 계획 구간 1
       → **행동 하나 계산에 4분**.
       reach는 80% 성공하지만 물체 조작은 약함. pick-and-place 한 번에 **1시간 이상** 소요
   - **V-JEPA 2-AC**
     - 샘플이 **10배 많은데도** 행동 하나에 **16초**, 모든 기술에서 더 높은 성능
   - 향후 개선 방향: 계획용 컴퓨팅 증설, 샘플·정제 횟수 축소, 세계 모델의 상상 속에서 feed-forward 정책을 학습해 계획 초기값으로 활용, V-JEPA 2-AC의 경우 **기울기 기반 계획** 가능성

			Grasp		Reach w/ Obj.		Pick-&-Place	
Method		**Reach**	Cup	Box	Cup	Box	Cup	Box
Octo (Octo Model Team et al., 2024)	Lab 1	100%	20%	0%	20%	70%	20%	10%
	Lab 2	100%	10%	0%	10%	70%	10%	10%
	Avg	*100%*	*15%*	*0%*	*15%*	*70%*	*15%*	*10%*
V-JEPA 2-AC (ours)	Lab 1	100%	70%	30%	90%	80%	80%	80%
	Lab 2	100%	60%	20%	60%	70%	80%	50%
	Avg	*100%*	*65%*	*25%*	*75%*	*75%*	*80%*	*65%*

Table 2 Zero-Shot Robot Manipulation

4.3 한계

- **Long-horizon planning(장기 계획)**
  - 자기회귀 예측은 오차 누적 → 롤아웃이 길수록 정확도 ↓
  - planning 구간이 선형으로 늘면 가능한 행동 궤적 수는 지수적으로 증가
    → 탐색 공간 폭발
  - 하지만 서브골 이미지 없이 pick-and-place 하려면 장기 계획이 필요
- **이미지 목표**
  - 현재는 시각적 목표(이미지)가 있어야 함
  - 실제 환경에서는 언어로 목표를 주는 것이 더 자연스러움
    → 잠재 세계 모델을 언어 모델과 정렬하는 연구가 향후 과제

5. Understanding, Prove 기반 분류

> 표현 공간 world model의 능력은 **학습된 표현 공간에 담긴 상태 정보에 의해 근본적으로 제한**
> → V-JEPA 2가 학습한 표현 자체 조사 + 시각 분류 과제에서 다른 비전 인코더와 비교
> ("그 인코더가 만드는 특징이 애초에 좋은 거 맞아?")

평가 과제

구분	특징	과제
움직임 이해	여러 프레임을 봐야 분류 가능 (사람의 제스처·동작 이해)	Something-Something v2 (SSv2), Diving-48, Jester
외형 이해 — 시간에 따른 변화	한 프레임만 봐도 대체로 분류 가능 (행동·장면·물체 인식)	Kinetics400 (K400), COIN, ImageNet (IN1K)

⭐ **V-JEPA 2는 움직임 이해 과제에서 최신 비전 인코더들을 능가하고, 외형 과제에서도 경쟁력이 있다**

Attentive Probe(평가용 분류기)

- 고정된 인코더 출력 위에 **4층 attentive probe 학습**
- 트랜스포머 블록 4개로 구성, 마지막 블록은 일반 self-attention 대신 학습 가능한 query 토큰을 쓰는 cross-attention

  — 영상에는 수천 개의 토큰 有 → 질문 토큰("이 영상은 뭐야") 1개로 전체 토큰을 훑어 보고 핵심을 뽑아 답을 내는 방식이라 효율적인 분류 O
- 추론 시에는 영상에서 고정 프레임 수의 클립을 여러 개 뽑아, 클립별 logit을 평균
- 해상도는 V-JEPA 2 사전학습 때와 비슷하게 유지

평가 프로토콜과 비교 대상

분류	모델	설명
이미지 자기지도	**DINOv2 (with registers)**	이미지 자기지도학습의 현재 SOTA
이미지-텍스트 대조학습	**SigLIP2**, **PEcoreG**	이미지-텍스트 대조 사전학습의 SOTA
비디오	**V-JEPA** (자기지도), **InternVideo2s2-1B** (비전-텍스트 대조 위주)	비디오 인코더

- 이미지 모델은 **각 프레임의 특징을 이어 붙여(concatenate)** 비디오에 적용

결과

Method	Param.	Avg.	*Motion Understanding* SSv2	Diving-48	Jester	*Appearance Understanding* K400	COIN	IN1K
Results Reported in the Literature								
VideoMAEv2 (Wang et al., 2023)	1B	–	56.1	–	–	82.8	–	71.4
InternVideo2-1B (Wang et al., 2024b)	1B	–	67.3	–	–	87.9	–	–
InternVideo2-6B (Wang et al., 2024b)	6B	–	67.7	–	–	88.8	–	–
VideoPrism (Zhao et al., 2024)	1B	–	68.5	71.3	–	87.6	–	–
Image Encoders Evaluated Using the Same Protocol								
DINOv2 (Darcet et al., 2024)	1.1B	81.1	50.7	82.5	93.4	83.6	90.7	86.1
$\text{PE}_{\text{core}}\text{G}$ (Bolya et al., 2025)	1.9B	82.3	55.4	76.9	90.0	88.5	**95.3**	87.6*
SigLIP2 (Tschannen et al., 2025)	1.2B	81.1	49.9	75.3	91.0	87.3	95.1	**88.0**
Video Encoders Evaluated Using the Same Protocol								
V-JEPA ViT-H (Bardes et al., 2024)	600M	85.2	74.3	87.9	97.7	84.5	87.1	80.0
InternVideo2_{s2}-1B (Wang et al., 2024b)	1B	87.0	69.7	86.4	97.0	**89.4**	93.8	85.8
V-JEPA 2 ViT-L	300M	86.0	73.7	89.0	97.6	85.1	86.8	83.5
V-JEPA 2 ViT-H	600M	86.4	74.0	89.8	97.7	85.3	87.9	83.8
V-JEPA 2 ViT-g	1B	87.5	75.3	90.1	97.7	86.6	90.7	84.6
$\text{V-JEPA 2 ViT-g}_{384}$	1B	**88.2**	**77.3**	**90.2**	**97.8**	87.3	91.1	85.1

Table 4 Action and Object Classification.

- **SSv2 (움직임)**: V-JEPA 2 ViT-g(256 해상도) **75.3** vs InternVideo 69.7 vs PECoreG 55.4
- **ImageNet (외형)**: **84.6**, V-JEPA 대비 **+4.6점**
- **6개 과제 평균**: 다른 비디오·이미지 인코더 중 **최고**
- **더 높은 해상도·긴 영상의 V-JEPA 2 ViT-g384**: 모든 과제에서 추가 향상, 평균 **88.2**

6. Prediction: Probe 기반 행동 예측

> **행동 예측(action anticiaption)** — 어떤 행동이 시작되기 전까지의 영상 클립을 보고 미래에 일어날 행동 맞히는 과제

📍이 과제를 위해 특별히 설계된 기존 SOTA 방법들보다, V-JEPA 2 표현 위에 **attentive probe만 얹어도** 크게 앞선

다.

과제 (Task)

EK100 데이터셋

- 45개 주방 환경에서 **1인칭 시점**으로 녹화한 요리 활동 **100시간**
- 각 영상에 행동 구간(시작·종료 시각, 행동 라벨)이 주석되어 있음
- 고유 행동 라벨 3,568개 = **동사** 97종 × **명사** 300종의 조합

예측 과제 구조

- 행동 시작 시각 **이전**의 영상 클립(context)을 **보고 명사, 동사, 행동(동사+명사 동시)**을 예측
- context의 끝과 행동 시작 사이의 간격 = **anticipation time (기본 1초)**
- 같은 context에서도 가능한 미래 행동이 여러 개일 수 있어서, 지표는 **mean-class recall-at-5** (상위 5개 예측 안에 정답이 있는지)

Anticipation Probe 구조

고정된 V-JEPA 2 **인코더와 예측기** 위에 attentive probe를 학습

1. 행동이 시작되기 1초 전에 끝나는 영상 클립을 샘플링해 **인코더**에 입력
2. **예측기**가 인코더 표현과 **1초 뒤 프레임에 해당하는 마스크 토큰**을 받아, 미래 프레임의 표현을 예측
3. 예측기 출력과 인코더 출력을 **토큰 차원으로 이어 붙여(concatenate)** attentive probe에 입력
4. 5장의 probe와 비슷하지만, 마지막 cross-attention이 query 토큰을 **1개가 아닌 3개** 학습
    - 각 query 출력이 서로 다른 선형 분류기로 가서 **행동 / 동사 / 명사**를 각각 예측
5. 분류기마다 **focal loss**를 독립적으로 계산해 합산한 뒤, 공유 attention 블록으로 역전파

비교 대상 (행동 예측 전용으로 학습된 3개)

모델	특징
InAViT	지도학습, 손-물체 상호작용을 명시적으로 모델링
Video-LLaMA	LLM(최대 70억 파라미터) 활용
PlausiVL	LLM(최대 70억 파라미터) 활용

결과 (Table 5)

설정: V-JEPA 2 ViT-L, ViT-H, ViT-g 모두 32프레임, 8fps, 256×256 해상도를 context로 사용. ViT-g384는 384×384.

모델	Action recall@5
ViT-L (3억)	32.7
ViT-g (10억)	38.0 (**+5.3**)
ViT-g384	39.7 (ViT-g 대비 **+1.7**)

- 모델 크기에 대해 **선형에 가까운 스케일링** 경향
- 더 높은 해상도의 context도 도움이 됨
- **3억 파라미터짜리 ViT-L조차** 80억 파라미터를 쓰는 이전 SOTA(PlausiVL)를 능가
- ViT-g384는 PlausiVL 대비 **+12.1점**, **44% 상대 향상**

정성적 결과 (Figure 11): 검증 셋 3개 샘플 (성공 2, 실패 1)

- 성공 사례: top-1이 정답일 뿐 아니라 top 2~5도 **맥락에 맞는 그럴듯한 행동**
    - 예: 정답이 "wash sink"일 때 "turn on water", "clean wall"도 수도꼭지와 벽이 보이니 타당한 후보. "rinse sponge"는 현재 진행 중인 행동이 1초 뒤에도 이어질 것이라 본 듯함
- 실패 사례: "close door", "put down spices package" 같은 그럴듯한 행동을 내놓지만 물체를 "tea package"로 정확히 맞히지 못함

한계점

- **완전히 해결한 것은 아님**: 동사, 명사, 또는 둘 다 틀리는 경우가 있음 (Appendix D.2에 분포 분석)
- **1초 예측에 국한**: 더 먼 미래를 예측하면 정확도가 떨어짐
- **주방 환경 한정**: 닫힌 어휘를 가진 EK100만 평가했으므로 다른 환경으로의 일반화는 미지수
- **고정된 행동 범주**: 학습 셋에 없는 행동 범주로는 일반화 불가

> 💡 V-JEPA 2는 사전학습 때 **익힌 미래 예측 능력** 덕분에, **작은 probe만 얹고**도
> 1초 뒤 요리 행동을 맞히는 과제에서 80억 파라미터 LLM 기반 SOTA를 44% 앞섰고, 모델이 클수록 성능이 올랐다. 다만 1초 예측, 주방 환경, 고정된 행동 범주라는 제약이 있다.

7. Understanding: VidQA(비디오 질의응답)

> VidQA : 열린 형식의 언어 질문에 답하는 것.
> V-JEPA 2가 VidQA도 가능한지 실험

구조

LLaVA 계열의 non-tokenized early fusion 방식

- 시각 인코더가 출력한 패치 임베딩 → LLM의 입력 임베딩 공간으로 투영(projection)해서 붙임
- MLLM은 end-to-end로 학습하거나, 시각 인코더를 고정한 채 학습 O

▼ MLLM

```
[V-JEPA 2] ← 이게 "시각 인코더" (영상을 이해하는 부품)
     ↓ (번역기로 연결)
[LLM]      ← 이게 "언어 뇌" (원래 텍스트만 다루던 부품)
     ↓
= 이 둘을 합친 전체 시스템이 "MLLM"
```

- 기존 VidQA용 MLLM의 인코더는 대부분 이미지 인코더(CLIP, SigLIP, Perception Encoder 등)를 프레임별로 독립 적용한 것.

  — 이미지-캡션 쌍으로 학습되어 언어와의 의미 정렬이 잘 되어 있어서 선택

⭐ 이 연구는 언어 지도 없이 사전학습된 비디오 인코더 — VidQA용 MLLM을 학습한 첫 사례

실험 구성

절	내용
7.2	통제된 설정(1,800만 샘플)에서 **V-JEPA 2와 다른 SOTA 인코더 비교**
7.3	같은 설정에서 **인코더 크기·입력 해상도 스케일링 효과**
7.4	전체 8,850만 샘플로 데이터를 늘려 **언어 정렬의 한계** 시험

7.1 실험 설정

평가 벤치마크

벤치마크	평가 대상
PerceptionTest	기억, 추상화, 물리, 의미 등 다양한 능력
MVP	물리 세계 이해. 최소 차이 영상 쌍으로 텍스트·외형 편향을 완화
TempCompass, TemporalBench, TOMATO	시간적 이해와 기억 능력
MVBench	일반 이해. 단일 프레임 외형 특징에 **편향**이 있음
TVBench	MVBench의 편향을 완화한 일반·시간적 이해 대안

Visual Instruction Tuning (3단계 점진적 학습)

단계	학습 대상	데이터
1단계	**projector만** 학습	이미지 캡셔닝
2단계	전체 모델	대규모 이미지 질의응답
3단계	전체 모델	대규모 비디오 캡셔닝·질의응답

- 설명
  - Projector는 보통 MLP, 시각 토큰을 LLM 입력으로 변환
  - 시각 인코더는 고정, 파인튜닝 모두 O
    - 고정 → 시각 특징의 품질을 더 깨끗하게 볼 수 있음
    - 파인튜닝 → 전체 성능이 더 좋아짐

      (두 설정 모두 실험)

7.2 이미지 인코더 비교

통제된 설정: 인코더만 바꾸고 나머지는 전부 동일하게 맞춤

- LLM: **Qwen2-7B-Instruct**, 시각 인코더 **고정**, 데이터 **1,800만 샘플**
- 비교 대상: V-JEPA 2(512×512 해상도로 사전학습), DINOv2, SigLIP-2, Perception Encoder

Method	Params Enc / LLM	Avg.	PerceptionTest SFT / Acc	MVP Paired-Acc	TempCompass multi-choice	TemporalBench (MBA-short QA)	TVBench Acc	TOMATO Acc	MVBench Acc
Off-the-shelf image encoders									
DINOv2 ViT-g$_{518}$	1.1B/7B	45.7	67.1	22.4	62.3	26.8	47.6	32.0	61.8
SigLIP2 ViT-g$_{384}$	1.1B/7B	48.1	**72.4**	26.2	66.8	25.7	48.7	33.2	64.0
PE ViT-G/14$_{448}$	1.9B/7B	49.1	72.3	26.7	67.0	27.5	51.6	34.0	64.7
V-JEPA 2 ViT-g$_{512}$	1B/7B	**52.3**	72.0	**31.1**	**69.2**	**33.3**	**55.9**	**37.0**	**67.7**

Table 6 Comparison between off-the-shelf image encoders and V-JEPA 2 in frozen encoder setting.

- 결과 설명
  - V-JEPA 2가 DINOv2, SigLIP, PE보다 **거의 모든 벤치마크에서 우수** (PerceptionTest만 SigLIP·PE보다 약간 낮음)
  - 특히 **MVP, TemporalBench, TVBench**(시간적 이해 중심)에서 향상이 뚜렷
  - 인코더만 바꿨으므로,
    언어 지도 없이 학습한 비디오 인코더가 언어 지도로 학습한 인코더를 능가할 수 있다는 증거가 됨 (기존 통념과 반대)
  - **이미지 인코더 대신 비디오 인코더**를 쓰면 시공간 이해가 좋아짐
    → 더 나은 비디오 인코더 개발이 필요함을 시사

7.3 인코더 크기와 입력 해상도 스케일링

> V-JEPA 2를 **3억 → 10억 파라미터**, 해상도 **256 → 512**로 키움

변화	개선 폭
3억 → 10억 (해상도 256 고정)	PerceptionTest **+0.9**, TVBench **+3.3**, MVBench **+1.2**
해상도 256 → 512	모든 과제에서 추가 향상. PerceptionTest **+2.2**, TemporalBench **+4.0**, TVBench **+3.3**

→ 인코더 해상도를 키우는 것이 유망한 방향임을 시사함

7.4 데이터 스케일링으로 SOTA 갱신 — PerceptionLM 8B 대비 개선 폭

정렬 데이터를 **1,800만 → 8,850만(4.7배)**으로 늘림

- **모델**
  - V-JEPA 2 **ViT-g384**, 프레임당 시각 토큰 **288개**.
    해상도가 높으면 LLM 입력에 시각 토큰이 많이 들어가는 부담이 있어서 이 설정을 선택
- **LLM 백본**
  - Llama 3.1, PerceptionLM(Cho et al., 2025)과 같은 레시피
- 학습 단순화를 위해 **풀링 없는 MLP projector** 사용

Method	Params Enc / LLM	Avg.	PerceptionTest Test Acc	MVP Paired-Acc	TempCompass multi-choice	TemporalBench (MBA-short QA)	TOMATO Acc	TVBench Acc	MVBench Acc
≤ 8B Video Language Models Results Reported in the Literature									
InternVL-2.5 (Chen et al., 2024)	300M/7B	52.1	68.9	39.9	68.3	24.3	29.4	61.6	72.6
Qwen2VL (Wang et al., 2024a)	675M/7B	47.0	66.9	29.2	67.9	20.4	31.5	46.0	67.0
Qwen2.5VL (Qwen Team et al., 2025)	1B/7B	49.7	70.5	36.7	71.7	24.5	24.6	50.5	69.6
PLM 8B (Cho et al., 2025)	1B/8B	56.7	82.7	39.7	72.7	28.3	33.2	**63.5**	**77.1**
V-JEPA 2 ViT-g$_{384}$ LLama 3.1 8B	1B/8B	**59.5**	**84.0**	**44.5**	**76.9**	**36.7**	**40.3**	60.6	73.5

Table 8 Comparison with state-of-the-art.

→ 데이터를 늘리자 성능이 고르게 향상되어 여러 벤치마크에서 SOTA를 달성

벤치마크	지표	개선 폭
PerceptionTest (test set)	정확도	**+1.3**
MVP	paired accuracy	**+4.8**
TempCompass	정확도	**+4.2**
TemporalBench (short-QA)	multi-binary accuracy	**+8.4**
TOMATO	정확도	**+7.1**

결론 — 비전-언어 정렬에는 **학습 데이터 규모**가 중요하며,
V-JEPA 2처럼 **언어 지도 없이 사전학습된 인코더도 충분한 규모에서는 SOTA에 도달**할 수 있음

5. Conclusion

> "인터넷 영상만 잔뜩 보여주고, 로봇 데이터는 아주 조금만 더해줬더니,
> **보고(이해) · 예측하고 · 계획해서 행동하는** AI가 만들어졌다."

1. 한계점

이번 논문의 한계 (4.3절에서 이미 언급)	여기서 제시한 해법 방향
장기 계획이 어려움 (오차 누적, 탐색 공간 폭발)	계층적 모델 — 큰 그림과 세부 사항을 나눠서 예측
이미지 목표만 가능 (언어 지시 불가)	언어를 V-JEPA 2-AC의 **"상상 공간"**으로 번역해주는 모델
10억 파라미터에서 멈춤	200억까지 키워본 다른 연구처럼 더 크게 스케일업

2. 결론

- joint-embedding predictive architecture
  → 물리 세계를 **이해·예측·계획**할 수 있는 world model로 이어질 수 O
- V-JEPA 2는 **움직임 이해가 필요한 행동 분류**와 인간 행동 예측(anticipation)에서 SOTA 성능 달성
- 대형 언어 모델과 정렬했을 때 **비디오 질의응답**에서도 기존 시각 인코더들을 능가
- V-JEPA 2의 표현을 이용해 사후학습한 행동 조건부 세계 모델 **V-JEPA 2-AC**는,
  실제 로봇으로 **Pick-and-Place 같은 파지 조작 과제를 zero-shot으로 성공**시킴

이러한 결과는 V-JEPA 2가 **환경을 효과적으로 지각하고 행동할 수 있는 고급 AI 시스템** 개발을 향한 한 걸음임을 보여줌